OPEN-SOURCE PRODUCT + CUSTOM BLOCKCHAIN

Decentralized Short-Video Network

Complete Project Summary & Blockchain Implementation Blueprint

A familiar social-video app.
Independently operated infrastructure.
A purpose-built chain for shared hosting commitments.



Prepared for Maxence Jules
October 12, 2026 | Version 1.0

Implementation proposal. Not a deployed system, audited protocol, investment offering or production performance claim.

READING GUIDE

Contents

Product definition, architecture, custom-chain creation, integration and operational release requirements in one implementation plan.

01 Executive summary	3
02 Final products and user experience	4
03 Scope and release boundaries	5
04 System architecture	6
05 Technology and reuse decisions	7
06 Blockchain design and consensus	8
07 On-chain state and custom modules	9
08 Transactions, execution and invariants	10
09 Creating the chain: build and genesis	11
10 Devnet, validator lifecycle and upgrades	12
11 Connecting the blockchain to the app	13
12 Hosting lifecycle and repair	14
13 Verification: honest claims first	15
14 Video processing and delivery	16
15 Identity, social records and portability	17
16 Discovery, feeds and peer assistance	18
17 Security and adversarial design	19
18 Moderation, privacy and legal readiness	20
19 Governance and open-source licensing	21
20 Deployment and day-to-day operations	22
21 Capacity and cost model	23
22 Transaction load and sustainable funding	24
23 Implementation roadmap and ownership	25
24 Testing and release acceptance	26
25 Repository and implementation contracts	27
26 Risks, decisions and launch checklist	28
A Glossary and interpretation	29
S Primary-source register	30

All requirements, architecture choices, workloads and test targets are proposals. Numbered source links identify the external facts on which the design is based.

PROJECT DEFINITION

01 Executive summary

Build an open-source short-video social platform with a familiar vertical feed, community-operated hosting, and a purpose-built blockchain that coordinates hosting commitments. The objective is to remove a single mandatory infrastructure owner, not to eliminate servers, bandwidth, operating costs, or accountability.

The final product has three parts: a viewer/creator application; software and a dashboard for independent network operators; and an open protocol with its own blockchain. Normal viewers do not need to buy cryptocurrency, approve blockchain transactions, or store other people's videos.

Recommended first release

Start with a responsive web app, moderated public videos, a small permissioned validator network, designated storage hosts, and ordinary HTTPS video delivery. Keep the source and deployment specifications open. Add optional peer-assisted delivery, alternative discovery services, and native mobile clients after the complete upload-to-playback path works.

Why create a blockchain?

The proposed chain provides a common record of provider registrations, storage obligations, accepted evidence and governance actions across independent operators. It is justified only if those operators need shared rules without trusting one database administrator. Cosmos SDK supports application-specific logic; CometBFT supplies the consensus engine. [S01, S05]

The project is not a new consensus algorithm, a video-inside-every-block system, or a token sale. Storage and network capacity remain separate resources. Content addressing alone does not keep a file online. [S12]

Decision	Baseline
Product	Short-video app, operator software, public protocol specification.
Blockchain	Independent Go application chain; SDK modules and CometBFT.
Network launch	Known validators and providers; honest disclosure of operator control.
Economics	Quotas and explicit sponsorship; no tradable token in the beta.
Success test	A second operator can run the software; playback survives loss of one host.

This document is a proposed implementation blueprint, not a deployed system or an audited protocol. Performance, total cost, user demand and production security have not been established.

PRODUCT DESIGN

02 Final products and user experience

The social application

The Home screen offers a chronological Following feed and a separate recommendation feed. Discover searches public creators, topics and videos. Create supports uploads, basic trimming, captions and publication status. Activity contains replies and notifications. Profile includes videos, account recovery, exports, privacy and hosting-status controls.

A creator sees Uploading, Processing, Review pending, Replicating and Published as distinct stages. Published means the app's moderation and hosting conditions are satisfied, not that a blockchain has certified the content as lawful or truthful. Availability degraded and Hosting renewal needed must be visible states.

The operator product

An installation wizard lets an operator choose storage/delivery, processing, account/discovery, or validator roles. The dashboard shows disk allocation, leases, job status, bandwidth, software updates, reports and service health. Capacity limits and an orderly leave-network workflow are required. A storage operator need not become a validator.

The developer product

Publish source code, versioned interfaces, schema files, test vectors, example clients and reproducible deployment manifests. Compatible clients should be able to choose independently operated services. The project does not become interoperable with ActivityPub, AT Protocol, IPFS or Filecoin merely by using similar concepts; each integration needs its own adapter and tests. [S14, S19]

User	Acceptance outcome
Viewer	Watch, follow, block and report without a wallet or compulsory peer sharing.
Creator	Upload, understand status, recover an account, export records and request removal.
Community operator	Install independently, set local rules and quotas, process appeals and hand off hosting.
Infrastructure operator	Accept bounded commitments, verify work, monitor costs and rotate service keys.
Developer	Build a second client against documented interfaces and a conformance suite.

Long-term vision: a normal social-video experience on top of a network that can outlive the original deployment. That outcome must be demonstrated by independent operation, not inferred from a node count.

PRODUCT DESIGN

03 Scope and release boundaries

Use a small invited community for the first beta. The initial audience and final brand remain open decisions; a developer or learning community is an example, not a validated market. Recruit willing creators and use only material they own or have permission to publish.

Release	Included	Explicitly deferred
Prototype	Login; upload; processed playback; four-node test chain; hosting records; failure demonstrations.	Public sign-up; monetary rewards; production assurances.
Community beta	Follows; comments; search; reports; blocking; recovery; independent operators; quotas; lease repair.	Open validator admission; unsupervised rewards; live video; commercial music library.
Version 1	Installable releases; documented governance; backup/upgrade drills; measured service targets; working exports.	A promise of TikTok-scale traffic or profitability.
Later extensions	Mobile clients; optional peer assistance; feed plugins; deeper portability; reviewed proof schemes.	Anything not funded, tested or supported operationally.

Proposed beta policy values

Limit uploads to 60 seconds and 100 MB of input per video, with one account allowance defined by its community. Start with 480p and 720p portrait-friendly outputs, do not upscale smaller inputs, and keep a separate cap on total output bytes. These are proposed constraints, not claims about maximum system capacity.

Use three operator-distinct copies of the accepted media package for publication, plus a separately managed creator/origin backup policy. Three copies are a beta test parameter, not a durability guarantee. Count failure domains as well as node identities.

Accessibility and safety requirements

Provide readable captions, accessible controls, keyboard navigation, reduced-motion settings, mute controls, and predictable playback. Include reporting, blocking and community contact information from the start. Apple and Google have user-generated-content requirements for apps distributed through their stores. [S22, S23]

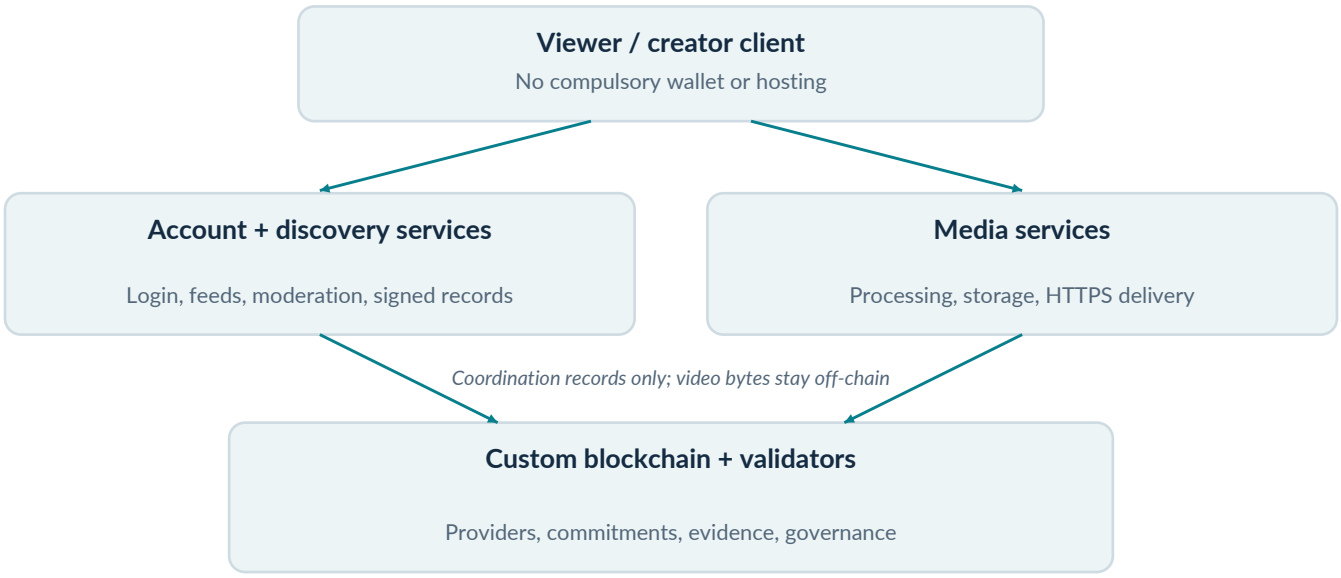
Explicit non-goals

Do not build a proprietary-code clone, put personal viewing histories on a public ledger, promise universal deletion, create a financial asset to fund an unmeasured system, or make every viewer a consensus participant. No native-app launch is necessary to establish the first engineering result.

ARCHITECTURE

04 System architecture

Separate consensus, social records and media transport. The blockchain is the coordination layer. Account hosts and indexes provide the social layer. Storage and processing nodes provide the media layer. Each has independent authentication, failure handling and operational limits.



Trust and ownership boundaries

A community controls admission, its account service and its moderation decisions. A provider controls its machine and the obligations it accepts. Validators order valid transactions; they do not automatically own videos or decide every community's editorial rules. The initial council controls the permissioned validator set and protocol parameters.

No mandatory commercial cloud

Provide a reference deployment on operator-owned or independently rented Linux machines. Optional commercial services may simplify deployments, but DNS, identity, media storage, search and transaction access must have documented replacement paths. A single mandatory gateway would remain a practical point of control.

The first beta can use signed provider manifests and HTTPS replication rather than public IPFS. Evaluate an IPFS adapter later only after privacy, removal and operational behaviour are tested. PeerTube demonstrates an existing HTTP/HLS plus optional WebRTC pattern; this project does not inherit its implementation automatically. [S14]

> **A chain outage should pause new final commitments, not force every existing video to disappear. Hosts may serve already approved content under their last valid policy until the applicable real-time hosting deadline; the client must disclose stale status.**

ARCHITECTURE

05 Technology and reuse decisions

Layer	Proposed baseline	Implementation responsibility
Web client	React + TypeScript	Player, uploads, accessible UI, operator dashboard.
App services	Python + FastAPI	Accounts, policies, social actions, feed API and moderation.
Database	PostgreSQL	Private account data, social records, job state and read models.
Jobs	Celery + RabbitMQ	Bounded durable processing and replication jobs.
Media workers	FFmpeg in isolated processes	Probe, transcode, thumbnails, captions and package checks.
Provider agent	Go + HTTPS	Storage inventory, signed endpoints, leases, replication and audits.
Chain	Go + Cosmos SDK + CometBFT	Deterministic modules, governance, queries and validator lifecycle.
Delivery	HLS/HTTPS first	Multiple hosts; optional WebRTC and TURN later.
Deployment	Docker Compose; Linux services	Reference devnet, node profiles, backups and diagnostics.

These are recommendations rather than confirmed dependencies of an existing repository. Pin exact versions and container digests, generate a software bill of materials, and record a licence decision for each dependency. Heavy media work belongs outside the FastAPI request process. [S17]

Blockchain compatibility and licensing

At the research date, Cosmos's release-family page lists SDK 0.55.x with CometBFT 0.40.x in its 2026.1 family. Recheck supported patch versions before implementation. Do not assume an independently selected higher version is compatible; the same page identifies CometBFT v1.x as unsupported. [S02]

Cosmos Enterprise PoA and Groups modules have a production-licensing boundary. Do not silently include them in the open-source baseline. A minimal custom authority module is proposed here and requires its own engineering and security review. [S01]

Reuse before replacement

Evaluate Loops for short-video UX, PeerTube for media/federation patterns, and Livepeer for distributed processing concepts. They are reference projects, not drop-in components of the proposed chain. Decide whether to adapt code only after licence and integration review. [S29, S14, S30]

BLOCKCHAIN CREATION

06 Blockchain design and consensus

Create a new application chain with its own chain ID, genesis state, validator set, modules and upgrade policy. Reuse the SDK's application framework and CometBFT's consensus rather than writing consensus from scratch. No EVM, general-purpose smart-contract runtime, bridge or IBC connection is required for the first release. [S01, S05]

Permissioned first, with explicit control

Use four equal-voting-power validators for the devnet. In the community beta, seek four genuinely independent operators; until then, describe actual control accurately. Consensus safety requires the relevant Byzantine-fault and network assumptions. With four equal validators, three form a greater-than-two-thirds quorum; one faulty validator is below one third, while two unavailable validators prevent new commits. [S05, S06]

Implement x/authority as the sole source of authorized validator membership and scheduled power changes. Do not simultaneously let a staking module overwrite the validator set. Validator updates must follow the selected CometBFT/SDK lifecycle and activation semantics; test this explicitly. [S06]

Resource control without a tradable token

For beta chain writes, accept only approved operator and service accounts. Require signatures, account sequences, explicit message authorization, bounded transaction size, execution gas metering and deterministic write quotas. Zero monetary fees do not mean unlimited computation. The custom admission/ante path must enforce the same consensus-critical rules during execution, not only in each node's mempool.

Viewers authenticate with the application. A service submits a creator's authorized hosting intent; providers sign their own acceptance. A transaction broadcaster is not an IBC relayer. It must not be able to manufacture provider acceptance or bypass a user's publication permissions.

Public-network gate

Permissionless validator or provider admission needs a separate Sybil-resistance and economic-security design, audited incentives, dispute rules and operational diversity. Increasing the number of anonymous identities is not sufficient. Do not describe the permissioned beta as trustless or permissionless.

> **Proposed launch policy: no token sale, no exchange listing, no mining and no automatic financial slashing. The first chain records bounded obligations and evidence, not speculative returns.**

BLOCKCHAIN IMPLEMENTATION

07 On-chain state and custom modules

Keep chain state compact and necessary for coordination. Use versioned Protocol Buffers for chain messages and explicit typed state. SDK modules provide state-owning keepers, message handlers, read-only query handlers and genesis hooks. [S03]

Module	State and responsibility	Representative messages
x/authority	Validator membership, council membership, scheduled changes, allowed submitters.	ProposeChange, ApproveChange, ExecuteChange.
x/providers	Provider ID, signing key, endpoint-manifest digest, capabilities, status, admitted operator group.	RegisterProvider, UpdateProvider, DisableProvider.
x/content	Opaque video ID, manifest digest, package bytes, submitting host, coordination status.	RegisterContent, WithdrawContent.
x/leases	Video/provider pair, terms digest, start and deadline, capacity reservation, status.	OfferLease, AcceptLease, RenewLease, EndLease.
x/evidence	Evidence mode, attestation/proof digests, signer set, challenge ID, round and result.	SubmitAuditBatch; reviewed proof messages later.
x/policy	Network-wide resource bounds and delayed configuration changes.	UpdateLimits through authorized governance.

Data that must stay off-chain

Video bytes, usernames tied to private details, email addresses, recovery secrets, individual watch events, complaint narratives, private messages and legal evidence belong outside the public ledger. Store endpoint details in signed off-chain manifests where possible. An opaque identifier or hash can still be linkable; hashing does not automatically anonymize personal data. [S13, S24]

Suggested primary keys and indexes

Use stable IDs for providers, content, leases, proposals and evidence batches. Maintain indexes for content-to-active-leases, provider-to-commitments, and deadlines-to-expiring-leases. Cap work per block and paginate all queries; do not sweep the entire catalogue in every block.

Separate content from its current hosting

A media manifest is immutable for a given content version. Hosting locations, terms and publication status are mutable records. A changed encoding produces a new package/manifest version; renewal of a lease must not silently change the bytes previously committed.

BLOCKCHAIN IMPLEMENTATION

08 Transactions, execution and invariants

Every state-changing message needs a named signer, authorization rule, replay protection, input bounds and deterministic state transition. Treat all transaction data as hostile. Chain execution must produce the same state from the same committed inputs. [S07]

Operation	Required checks	State change
RegisterContent	Approved host; user intent accepted by that host; valid digest; byte limit; unused content version.	Create pending record; no hosting claim yet.
Offer / AcceptLease	Valid content; eligible provider; provider signature on terms; unexpired offer; declared capacity bound.	Reserve stated bytes; create active obligation.
SubmitAuditBatch	Allowed auditor; valid signature; correct epoch; accepted schema; unused evidence ID.	Record attestation, not an unqualified storage proof.
Renew / EndLease	Authorized party; valid state; monotonic term version; legal transition.	Extend agreed obligation or close it and release reservation.
Governance action	Unique approvals; threshold; expiry; delay; valid resulting validator/parameter set.	Schedule and execute an auditable change.

Mandatory invariants

A provider cannot accept the same content version twice to count as two replicas. Declared reserved bytes cannot exceed its declared admitted capacity. Lease and governance transitions cannot be replayed. Byte counters cannot underflow or overflow. Expired evidence cannot renew current status. Withdrawal stops new sponsored leases. These checks constrain accounting; they do not establish that physical capacity or operator independence is truthful.

Deterministic execution discipline

Do not fetch videos, call an external API, run FFmpeg, read local wall-clock time, use floating-point scoring, or iterate an unordered map to construct consensus state. Use committed chain context, integers, explicit sorting and bounded work. External observations arrive as signed transaction inputs whose meaning is specified. [S07]

Commit acknowledgement

A successful broadcast or mempool response is not a committed successful transaction. The broadcaster waits for inclusion and execution success, then updates the application projection. Persist the transaction hash, message ID and result; reconcile after timeouts before retrying the business action.

BLOCKCHAIN CREATION

09 Creating the chain: build and genesis

Use an official example application to learn the scaffold, then remove unused functionality deliberately. The reference quickstart includes modules that are not part of this proposal; a tutorial chain is not the final architecture. [S08]

Build sequence

Step	Concrete output
1. Pin the stack	Choose an active compatible SDK/CometBFT family; record exact commits, Go toolchain, licences and container digests.
2. Create chain application	Create the Go module, app wiring, binary name, address configuration, API routes and test scaffolding.
3. Define protocol types	Write tx.proto, query.proto, state.proto and genesis.proto for each custom module; generate code and review changes.
4. Implement state	Write keepers, authorization, message/query handlers, genesis validation, events, migrations and tests.
5. Wire authority	Connect the approved validator set to initialization and scheduled updates; remove competing validator-selection paths.
6. Create genesis	Generate public parameters and initial authorities; include no private keys or shared test secrets.
7. Start and verify	Run local validators, submit one hosting transaction and verify equal committed application hashes.

Genesis fields to specify

Define chain_id (example: shortvideo-dev-1), genesis timestamp, validator consensus public keys and voting powers, application account addresses, council keys and threshold, admitted service/provider accounts, empty content and lease state, resource limits and protocol versions. The final JSON schema depends on the selected application and SDK; this is a required field inventory, not a ready-made genesis file.

Keys and network material

Generate validator consensus keys locally for each operator. Keep peer identity, validator signing and governance keys separate. Distribute a reviewed genesis file and its cryptographic digest, public peer endpoints and a signed release manifest. Never copy one validator signing key across simultaneously active nodes.

Definition of a created blockchain

A new chain exists when independent node processes initialize from the same reviewed genesis, agree on committed application state, accept only valid project transactions and recover consistently from restart. A renamed binary or a token deployed on another chain does not meet that definition for this project.

BLOCKCHAIN CREATION

10 Devnet, validator lifecycle and upgrades

Local development topology

Run four validator containers with distinct data volumes and peer identities, one query/index service, two app-service deployments, and three storage agents. One physical workstation is acceptable for development but is one failure domain. Use separate machines and operators for the independence tests.

Required devnet demonstrations

Register a content manifest, accept three hosting commitments, replay the committed event stream into the application database and show the same result after restart. Stop one validator and observe continued commits under the test assumptions. Stop two and verify that no new commitment is presented as final. Restore nodes without resetting the network or fabricating history.

Validator admission and removal

A council action specifies a public key, voting power, activation point and reason digest. x/authority validates the resulting set and emits updates only at the prescribed lifecycle stage. Add tests for key rotation, removal, duplicate keys, nonpositive total power and changed quorums. The ABCI specification governs when updates become effective. [S06]

Upgrading without losing state

Publish a versioned migration proposal, reviewed source, release hashes and a scheduled activation point. Rehearse on a copy of testnet state. Compare pre/post invariants and restart tests, then coordinate operator installation. Do not assume an old binary can safely read new state after an irreversible migration.

Recovery and signing safety

Back up chain state and signer safety metadata according to the selected signer implementation. When replacing hardware, stop and verify the old signer before activating the replacement. Keep query/public API nodes separate from signing endpoints. A stalled network needs a documented operator recovery procedure, not an application-level override of finality.

> **Initial policy proposal: three of four council members approve ordinary network changes, with a review delay. Council voting is application governance, not the same process as consensus voting. A chain-wide stall may prevent on-chain governance itself from executing.**

INTEGRATION

11 Connecting the blockchain to the app

Keep chain access behind a well-defined adapter. Python services request bounded operations through an operator-authorized Go broadcaster or generated client. Read models come from committed events plus periodic queries. The web client does not need to track every block.

Interface	Proposed contract
Client API	POST upload-intents; GET upload status; POST publish; GET feeds; POST reports; POST removal; GET exports.
Provider API	Signed capability/endpoint manifest; accept offer; fetch package; return transfer receipt; report inventory.
Chain commands	Register provider/content, accept/renew/end lease, submit evidence and execute authorized changes.
Chain queries	Provider status, content manifest digest, active leases, deadlines, evidence and governance state.
Events	ContentRegistered, LeaseAccepted, LeaseRenewed, LeaseEnded, EvidenceAccepted, PolicyChanged.

Idempotency across systems

Use a transaction outbox in PostgreSQL for intended work. The broadcaster records attempt IDs and reconciles unknown submissions before resending. Use a stable business ID so one successful action cannot be duplicated by a retry with a new transaction hash. Providers enforce the same offer/lease idempotency locally.

The indexer persists a last-processed block height and event identity in the same database transaction as its projection updates. Restart by backfilling from the checkpoint. Periodically reconcile active leases from canonical chain queries. The database projection is a cache of coordination state, not a substitute ledger.

Multiple gateways and clear trust

Allow users and operators to configure more than one service endpoint. At beta, disclose that the chosen gateway is trusted for read responses. Stronger clients can verify supported chain state proofs against trusted light-client headers; comparing two unauthenticated HTTP responses is not an equivalent security guarantee.

Pending and degraded states

The UI separates requested, submitted, committed and failed operations. If the chain stops, queue requests with explicit expiry and show them as pending. Do not activate a new storage promise based only on a local database write. Existing approved playback may continue from reachable hosts under the last accepted terms.

STORAGE NETWORK

12 Hosting lifecycle and repair

Give every public media package a content version and a manifest listing renditions, segment hashes, byte totals and format metadata. Store the full manifest off-chain and its digest on-chain. Nodes verify received bytes against the manifest before advertising the package.

```
OFFERED -> ACCEPTED -> ACTIVE -> RENEWED
                         |           |
                         +-> AT_RISK +-> ENDED
                         +-> EXPIRED
Content: PENDING -> PUBLISHED -> WITHDRAWN
```

Provider selection

An off-chain scheduler proposes hosts using available declared capacity, compatible policies, measured reachability and operator/failure-domain diversity. Providers must explicitly accept. The chain verifies admissibility and accounting, not the physical truth of a declared region or independent owner. Keep a reviewed operator registry for the beta.

Replication and publication

Transfer the package over authenticated HTTPS. Require checksum verification and provider-signed acceptance of exact terms. Publish only after the community approves the content and the target number of active commitments exists. Display the verification mode separately: provider acceptance, auditor attestation or a future reviewed proof scheme.

Detect, repair and drain

An off-chain monitor tests reachability and retrieval. If a host fails, mark its local availability degraded, select a replacement, transfer from a surviving copy and confirm a new commitment. Do not terminate the last reachable copy before replacement. A host leaving voluntarily stops new offers, transfers accepted work and waits for handoff or the agreed deadline.

Time, expiry and chain stalls

Use explicit real-time hosting deadlines in lease terms; do not convert block count to a guaranteed number of seconds. Chain transitions use committed timestamps/context. Hosts apply the agreed real-time deadline locally and show clock uncertainty. If consensus stops, on-chain expiry can lag real time; offline service extensions are not final on-chain renewals.

Retention must be actively supported. IPFS pinning is one optional implementation mechanism, not proof of permanent storage or guaranteed retrieval. Backups, availability and cryptographic integrity are separate concerns. [S12]

STORAGE VERIFICATION

13 Verification: honest claims first

The initial beta uses signed provider receipts and independently operated audit services. Label those records as attestations. They are not trustless proof of storage, operator independence, lawful content, or future service quality.

Question	Beta evidence	Remaining limitation
Did the provider accept?	Signature over package digest, bytes and terms.	Acceptance is a promise, not possession.
Could data be retrieved?	Auditor performs sampled and periodic full downloads; verifies hashes.	Auditor can lie; provider may discriminate or fetch elsewhere.
Are there separate replicas?	Reviewed distinct operators and failure domains.	Multiple node IDs do not prove independent copies.
Is playback good?	Real client and external probes measure startup, throughput and errors.	A storage check is not a playback-service guarantee.

A later cryptographic verification track

Filecoin PDP commits to stored pieces using Merkle trees, uses randomized challenges and verifies submitted inclusion proofs on Filecoin. It is a useful reference, not a library that automatically gives a new Cosmos chain the same properties. [S10]

Choose either an explicit external Filecoin integration with its trust/availability dependencies, or a separately reviewed local scheme. For a local scheme, specify chunking, commitment format, challenge randomness, deadlines, replay resistance, proof-size bounds and deterministic verification. An external randomness beacon adds signature-verification and liveness requirements; validators must not fetch it independently during execution.

What not to claim

Do not call a plain Merkle inclusion proof proof of a uniquely stored replica. Filecoin distinguishes Proof of Replication from continued-storage proofs. Distinct identity, physical redundancy and low-latency retrieval require additional arguments. [S11]

Test challenge predictability, cached proofs, outsourcing, colluding providers and selective service. Even correct sampled proofs provide only the guarantee of their construction and threat model. Delay automatic rewards and financial penalties until evidence rules and failure handling have independent review.

> **Launch gate: I cannot confirm a secure permissionless storage-proof system from this architecture alone. That requires a specified construction, implementation, adversarial testing and external review.**

MEDIA IMPLEMENTATION

14 Video processing and delivery

Upload and quarantine

Authorize the upload before receiving bytes. Enforce account quotas, content length, duration, dimensions and total processing budget. Store untrusted input in quarantine under generated names; check actual file structure rather than trusting its extension or declared MIME type. Apply layered validation and least-privilege storage access. [S18]

Worker pipeline

A queue assigns the job to an isolated worker. Probe input, generate supported renditions and thumbnails, normalize orientation, preserve synchronization, and attach user-provided or reviewed captions. Use CPU, memory, disk and wall-time limits, no unnecessary outbound network, a read-only runtime and a disposable workspace. Run as an unprivileged user.

Hash the produced segments and write a versioned manifest. Deterministic consensus does not require separate encoders to produce byte-identical output. Instead, the chosen output is inspected and committed by digest. A paid processing network would need a separate correctness/quality verification design; it is deferred.

Playback path

Serve an HLS master playlist and rendition playlists over HTTPS from participating hosts. Support multiple candidate hosts, bounded retries, byte integrity checks where the client fetch path allows them, and a fallback profile. HLS supports adaptive streaming using standard HTTP infrastructure. [S16]

Keep prefetch bounded to avoid downloading many unwatched videos. Cancel obsolete requests when the viewer scrolls. Record startup delay, playback failures, rebuffering and bytes actually delivered, with privacy-preserving aggregation. A successful HTTP response alone is not a completed watch.

Failure and deletion behaviour

Make jobs idempotent. A worker crash leaves a resumable or retryable job, never an apparently published video. Use staging prefixes and an atomic publication transition. Garbage-collect abandoned uploads and failed renditions. Removal propagates to approved hosts and indexes; independently retained copies remain outside the operator's technical control. [S13]

Record FFmpeg build flags, included codecs and licensing obligations. FFmpeg's licence changes with optional GPL components, and open-source licensing does not settle every codec patent issue. [S26]

APPLICATION IMPLEMENTATION

15 Identity, social records and portability

Keep login separate from chain signing

Use a reviewed authentication implementation with passkey support where practical and a secure recovery path. A web passkey is not assumed to be a Cosmos transaction key. Keep email, recovery information and device sessions off-chain. Operators, not normal viewers, hold the beta chain-submission credentials.

Signed, versioned social records

Use stable account IDs rather than usernames as primary keys. A proposed public-record envelope contains schema version, author ID, record ID, sequence, previous-record digest, key ID, payload and signature. Specify which records are public; saved videos, private preferences and raw viewing telemetry are private by default.

For JSON records, adopt a tested canonicalization profile rather than inventing one. RFC 8785 describes repeatable JSON canonicalization for cryptographic operations and is an Informational RFC. Represent large sequence values safely and publish cross-language test vectors. [S20]

A practical portability progression

In beta, the account host can hold a delegated signing key; disclose that custody and the need to trust the host. Provide encrypted backups, exports and a verified migration procedure. Later, a user-controlled root identity can delegate to replaceable hosts, subject to recovery and key-rotation tests. Changing hosts does not guarantee every outside follower service immediately updates its records.

AT Protocol shows that independent identities, personal data servers, feeds and indexes can be separate services. This is a reference pattern, not automatic compatibility or a feature supplied by the blockchain. [S19]

Recovery and moderation interactions

Require strong confirmation for recovery-key changes, session revocation and host migration. Preserve an auditable authorized key history without exposing secrets. Deleting an account revokes future actions and starts removal according to the retention policy. A user's export does not include another person's private data or confidential moderation evidence.

Off-chain database entities

Accounts, sessions, key delegations, public records, videos, renditions, social edges, comments, moderation cases, upload jobs, outbox entries, chain checkpoints and aggregate metrics are separate entities. Apply per-object authorization and retention rules. Never treat a ledger inclusion as permission to access private application data.

APPLICATION IMPLEMENTATION

16 Discovery, feeds and peer assistance

Make discovery replaceable

A feed response contains stable content IDs, display metadata and candidate playback locations. Keep search and feed indexes rebuildable from permitted records and hosting state. Expose versioned feed interfaces so another community can offer a compatible discovery service without running the original private backend.

Start with understandable recommendations

Use candidate generation, scoring and re-ranking as a simple structure. This is a documented general recommender pattern. [S21] Select candidates from follows, chosen topics and recent approved uploads; rank with explicit rules; then filter blocked accounts, withdrawn videos and repeated creators.

Keep a chronological Following feed and a clear Not interested control. Treat relevance, saves, voluntary return and negative feedback as evaluation signals. Do not optimize exclusively for watch time. Avoid publishing exact anti-abuse thresholds that would undermine protection, while documenting the ranking factors and user controls.

Peer-to-peer is an optimization, not the only route

Add optional WebRTC segment sharing after HTTPS playback meets the agreed targets. PeerTube demonstrates a player that combines HTTP segments from hosts/mirrors with segments from peers. [S14] Implement bandwidth and battery limits, a visible opt-in and an immediate fallback.

Direct connections may not work through every network. TURN can relay traffic and can be self-hosted; that relay consumes real bandwidth. [S15] Choose an origin download over a relay when it is better for reliability or cost. Test mobile/background behaviour rather than assuming continuous sharing.

Privacy and moderation boundaries

Do not enable public IPFS advertising or browser peer sharing for private content. P2P metadata may reveal node identity and requested content; encryption in transit is not anonymity. [S13] The client must apply current local moderation decisions before selecting sources, including peer sources.

> **Measure peer-supplied bytes separately from total payload, host traffic and relay traffic. A high peer percentage does not by itself prove lower total cost, better privacy or better playback.**

SECURITY

17 Security and adversarial design

Threat	Required control	Test evidence
Malicious uploads	Isolated workers; quotas; format validation; bounded CPU/disk/time.	Malformed corpus, resource-exhaustion and sandbox tests. [S18]
Key compromise	Separate service/validator/governance keys; least privilege; rotation; offline recovery.	Revocation and controlled recovery drill.
Chain spam / state bloat	Allowed submitters; bounded fields; gas; quotas; paginated storage and event processing.	Flood and oversized-proof tests with predictable rejection.
Colluding providers	Operator review; diverse hosting; external retrieval checks; no unreviewed rewards.	Outsourcing, multiple identities and false receipt scenarios.
Gateway or index tampering	Signed records; verified digests; alternate endpoints; explicit read trust.	Bad manifest, stale record and forged event tests.
Validator attack / partition	Protected signing; independent operators; peer controls; deterministic transitions.	Fault and partition tests with no fabricated finality.
Supply-chain compromise	Pinned dependencies; signed builds; review; isolated CI secrets; SBOM.	Clean build, dependency audit and secret scanning.

Separate data from executable instructions

Treat titles, comments, captions and remote manifests as untrusted input. Never interpolate them into shell commands, database queries, administrative instructions or model prompts with execution privileges. Any future AI moderation or support component must not gain authority to change chain policy or execute tools based on user content.

Abuse protection for non-media endpoints

Rate-limit login, search, uploads, reporting and provider discovery. Defend against SSRF when fetching remote media or endpoint manifests: constrain schemes, redirects and permitted destinations; reject private-network targets unless explicitly configured by the operator. Keep internal RPC, queue and database ports private.

Release gate

Require security review of chain authorization, authority updates, custody, proof verification and sandbox boundaries. Use maintained dependencies and a documented vulnerability channel. Cosmos publishes a security and maintenance policy; using its framework does not audit the custom modules. [S09]

TRUST AND SAFETY

18 Moderation, privacy and legal readiness

Moderation is an operating function

Provide clear community rules, report and block actions, an administrator queue, decisions with reasons, appeals and an urgent escalation process. Keep sensitive case evidence access-controlled and off-chain. A signed public label can express a community's decision without exposing the complainant.

Apply policy at account hosts, discovery services, storage admission and clients. Operators retain responsibility for what their own services distribute. Validators do not determine the legality of every video. PeerTube offers a precedent for local operator moderation; Apple and Google impose UGC controls for their app stores. [S31, S22, S23]

Removal must have precise promises

A removal action delists content, cancels or ends participating hosting commitments according to policy, purges controlled caches and revokes relevant credentials. Preserve only what is legally justified under a documented retention rule. Do not promise erasure of copies retained by independent parties, or deletion of all historical public-ledger records. [S13]

Privacy assessment before public launch

Map the personal information each role receives, its purpose, access, retention and location. Keep raw watch histories out of public protocols. Publish what node operators can observe. Minimize telemetry and give meaningful controls over personalized feeds and peer contribution.

For a Canadian or Quebec operation, obtain current professional advice on applicable privacy rules, cross-border processing, age-related requirements and incident duties. The OPC overview explains that applicability depends on activities, organization and data flows; its cited page is dated 2018 and is not a current-law checklist. [S24]

Rights and distribution

Use original branding and a creator upload licence that covers hosting, replication, transcoding and playback. Copyright covers software, music and audiovisual works; a hash or timestamp does not establish ownership or permission. [S25] Defer commercial music, payments and native-store launch until their respective rights and policy reviews are complete.

> **No app-store approval, legal compliance, global deletion guarantee or safe deployment involving minors can be confirmed from this blueprint alone. These require operational policies and jurisdiction-specific review.**

GOVERNANCE

19 Governance and open-source licensing

Three separate kinds of authority

Protocol governance controls validator admission, resource bounds and upgrades. Community governance controls local membership and moderation. The project's maintainers control what code enters the reference repository. Do not conflate any of these with ownership of users' content.

Beta governance proposal

Use four named council seats with a three-approval threshold, one approval per authorized seat, expiry and a published delay for ordinary changes. Implement this as an auditable project module or reviewed compatible component. Do not assume the commercially licensed Enterprise Groups module is available under the desired terms. [S01]

Document who can pause new admissions, how keys are replaced, what happens when the council loses quorum, and how operators choose upgrades. Emergency application actions may disable a compromised endpoint locally; they cannot retroactively invent a committed chain decision. Publicly record material control changes.

Proposed licence split

Consider AGPLv3 for original server and application code where network-use source reciprocity is wanted, and Apache 2.0 for original protocol client libraries where permissive reuse is preferred. AGPL's relevant source obligation for modified network-interactive versions is in section 13; Apache has its own notice and patent terms. Confirm compatibility for actual combinations. [S27, S28]

A protocol specification, contributed code and third-party dependencies need explicit licences. Retain original notices and do not relicense a fork beyond the rights granted. FFmpeg build choices require their own licence record. Source availability alone does not establish an open-source licence. [S26, S27, S28]

Community contribution system

Publish a contribution guide, code of conduct, security-reporting instructions, review rules and a decision log. Require tests for protocol changes, migration notes for state changes and reproducible evidence for performance claims. Assign maintainers for chain, media, application, infrastructure and trust/safety.

> **The beta is a permissioned community network with open-source software. An independently governed permissionless network is a separate future release, not a label to apply before its controls exist.**

OPERATIONS

20 Deployment and day-to-day operations

Deployment profiles

Provide a one-machine development profile and separate production-oriented profiles for account services, validators, storage agents and workers. Every profile documents persistent volumes, secrets, inbound ports, health checks, backup requirements and upgrade steps. Avoid running untrusted transcodes beside validator signing material.

Initial host sizing is a benchmark task

Do not advertise universal minimum hardware. Measure memory, CPU, disk I/O, database growth, encoding throughput and network capacity under the intended workload. Record the exact hardware, codec settings and concurrency with every benchmark. A small devnet working on one workstation establishes neither production capacity nor operator independence.

Monitoring and on-call responsibilities

Area	Monitor	Required response
Blockchain	Commit age, validator participation, peer reachability, state growth.	Distinguish quorum loss from app/index lag.
Storage	Reachable replicas, inventory mismatch, lease deadlines, repair backlog.	Repair from surviving data and notify affected creators.
Media	Processing age, failure rate, CPU/disk limits, output sizes.	Quarantine faults; retry safely; cap intake.
Playback	Startup delay, rebuffering, errors, bytes and fallback use.	Identify host/network failures without exposing raw histories.
Trust and safety	Unreviewed reports, urgent escalations, appeals.	Named coverage and incident process.

Backups and recovery

Back up application databases, signed record exports, configuration and critical media independently. A replicated live object is not automatically a recoverable historical backup. Test restoration with the origin offline. Preserve signer safety and never activate a duplicate live validator identity.

Chain stall and end-of-service procedure

Allow approved reads and playback within existing terms, but clearly freeze new final commitments. When a community closes, notify creators, provide exports, stop new leases, assist host migration and purge controlled data under policy. Publish the effect of any unfunded leases rather than silently presenting content as permanently available.

CAPACITY PLANNING

21 Capacity and cost model

The following scenarios are arithmetic planning examples, not measured demand, performance or vendor quotes. Use decimal MB/GB/TB. Assume 20 viewing minutes per daily active viewer, 2 Mb/s delivered bitrate, 30 days, a 25 MB stored package per upload and three full stored copies.

Illustrative load	Daily viewers	Uploads/day	Viewer payload/month	New 3-copy storage/month
Small beta	100	20	900 GB	45 GB
Growing community	1,000	200	9 TB	450 GB
Larger scenario	10,000	2,000	90 TB	4.5 TB

```
Viewer payload GB/month = DAU * minutes/day * 60
                          * bitrate_Mbps / 8 / 1000 * days
New stored GB/month = uploads/day * package_MB
                      * replica_count * days / 1000
```

The package assumption includes retained media outputs for this calculation; add separately retained originals, thumbnails, manifests, chain/database state and backups. Existing retained uploads accumulate. At steady state, retention duration and replacement policy determine total inventory, not just monthly growth.

Traffic does not vanish with peer delivery

Separate viewer payload, replication, repair, prefetch, protocol overhead and TURN relay traffic. Peer sharing shifts upload responsibility among participants; it does not eliminate viewer downloads. Let p be the measured peer share of viewer payload, then estimate host-served viewer payload as (1 - p) times total, before extra traffic and double-counted relay hops.

Actual budget worksheet

Monthly cost = fixed machines + average billable storage GB-months × storage rate + billable transfer GB × transfer rate + processing hours × processing rate + monitoring/backups + operational labour + trust/safety + allocated security/legal work. Avoid double-counting resources already included in a fixed-server quote.

Collect comparable quotes in one currency, preferably CAD for planning, including transfer allowances, taxes and overage rules. Model total network cost separately from the founder's cash outlay and from volunteer contributions. I cannot confirm an actual monthly operating budget without traffic measurements, deployment choices and provider quotes.

CAPACITY AND ECONOMICS

22 Transaction load and sustainable funding

Size coordination independently of video

An illustrative upload with three providers could produce one content registration and three acceptance transactions. At 200 uploads per day this is 800 transactions/day, or about 0.0093 transactions/second on average, before offers, renewals, audits, governance and bursts. This is a workload estimate, not chain throughput.

Audit every retained lease separately each day and the load scales with the retained catalogue. For 200 uploads/day, 90 days and three copies, there are 54,000 lease audits/day if no content leaves earlier. Batching observations can reduce transaction count but not remove the work of checking data or retaining verifiable evidence.

Measure proof and state costs

Record bytes and gas per transaction, peak batches, verification CPU, application-state growth, historical log growth and catch-up time. Keep proofs, attestation batches and query responses bounded. A digest on-chain reduces ledger bytes only if the corresponding evidence remains available where needed.

Fund storage before promising it

In beta, a community explicitly sponsors an upload allowance and lease duration. Hosts volunteer or have a separately agreed service arrangement. Possible later revenue includes managed hosting, community subscriptions, support and creator storage plans. These are business hypotheses, not forecasts or validated willingness to pay.

Keep any non-cash contribution credits inside a clearly bounded service policy. Do not promise that credits are money, an investment, or sufficient compensation for actual costs. Do not award credits simply for self-reported storage, fabricated views or identity count.

Token decision gate

A tradable token requires its own purpose, security model, issuance, treasury controls, economic abuse analysis and current legal/tax review. It is not necessary to create the proposed application chain. Defer it unless a demonstrated coordination need cannot be handled more simply.

> **Financial sustainability test: every accepted hosting promise has a named responsible party, a known term, a resource budget and a renewal or exit path. A token price is not a substitute for those commitments.**

DELIVERY PLAN

23 Implementation roadmap and ownership

Plan by evidence-based milestones rather than a fixed launch date. A solo developer can lead the prototype; public operation also needs reliable operator participation, security review and moderation coverage. No staffing commitment or completion date has been established.

Stage	Primary work	Exit evidence
0. Requirements	Choose initial community, threat model, data policy, licence and architecture decisions.	Reviewed scope, policy values, acceptance tests and source/version register.
1. Media vertical slice	Login, authorized upload, isolated processing, HTTPS playback and removal.	One permitted video completes the lifecycle; worker crash recovery works.
2. Custom chain	Implement authority, provider, content and lease modules; genesis and four-node devnet.	Valid transactions commit; invalid/replayed ones fail; restart and quorum tests pass.
3. Distributed hosting	Provider agent, signed receipts, scheduler, replication, repair and chain adapter.	Uploader/origin can disconnect; another host serves verified bytes.
4. Community beta	Feeds, follows, comments, reports, recovery, second operator and observability.	Moderated real usage with measured costs and documented trust boundaries.
5. Release hardening	Security review, upgrades, backup restore, installer, conformance suite and release signing.	Independent install and failure demonstrations meet launch gates.
6. Expansion	Optional P2P, native apps, deeper portability or reviewed proofs.	Each feature passes its own benefit, security and operating-cost tests.

Critical dependency path

Media package definition precedes hosting commitments. Authority and lease invariants precede financial consequences. Removal policy precedes public replication. Account custody and recovery precede portability claims. Real retrieval measurements precede peer-delivery or proof-system optimization.

Required ownership

Assign one accountable owner each for product/application, chain/security, media/provider agents, deployment/on-call and trust/safety. One person may initially cover several roles, but the responsibilities and backup coverage must still be explicit. Seek independent review for consensus-critical code and legal/privacy launch decisions.

QUALITY ASSURANCE

24 Testing and release acceptance

All numeric targets below are proposed beta gates, not achieved benchmarks or promised service levels. Publish the test environment, input corpus, concurrency and measurement definition with results.

Test	Proposed gate / evidence
Consensus correctness	Equal application hashes after deterministic replay; reject forged, unauthorized and replayed messages.
Quorum behaviour	Four equal validators: stop one and sustain commits; stop two and show stalled writes without false success.
Hosting resilience	Three designated hosts: remove one and the origin; retrieve from survivors and complete verified repair.
Integrity	Corrupt a segment or manifest; detect mismatch before accepting a replica or presenting verified status.
Playback	Controlled 20 Mb/s downlink and 50 ms RTT: p95 startup <= 3 s and rebuffer ratio <= 1%; report device, loss and sample size.
Processing	A 60-second supported input reaches ready-for-review in <= 120 s at the declared worker concurrency and hardware.
App targets	Invited beta design point: 100 simultaneous viewers and 10 simultaneous upload sessions; measure rather than assume capacity.
Idempotency	Repeat the same business operation, timeout and restart components: one final transition and no leaked reservation.
Recovery	Restore database, media and index on clean hosts; verify credentials and signer safety; document RPO/RTO measured.
Moderation/removal	Controlled-service delisting within 5 min; deletion workflow verified; outside-copy limitation visibly disclosed.
Operator independence	A second operator installs without private setup instructions and serves the agreed test catalogue.

Testing layers

Use unit and property tests for state machines, fuzzing for parsers/proofs, protocol contract tests across languages, integration tests with real stores and queues, end-to-end browser tests, load tests and controlled fault injection. Test migrations against a representative state snapshot.

Permissionless expansion requires more

Add adversarial providers, Sybil identities, selective retrieval, outsourced proofs, economic manipulation, gateway censorship and compromised signing keys. Commission review of the actual implementation and threat model. Passing beta gates does not demonstrate security against an unrestricted economically motivated network.

ENGINEERING HANDOFF

25 Repository and implementation contracts

Use a monorepo initially so protocol, chain, API and client changes can be reviewed together. Keep generated code separate from handwritten logic, and make every protocol version reproducible.

```
shortvideo-network/
  apps/web/                  # viewer + operator UI
  services/api/              # FastAPI and social data
  services/indexer/          # committed chain projections
  services/broadcaster/      # authorized transaction sender
  chain/app/                 # SDK application wiring
  chain/x/{authority,providers,content,leases,evidence,policy}/
  proto/                     # versioned chain/service types
  agents/provider/           # Go storage and delivery agent
  workers/media/             # isolated media jobs
  packages/protocol/         # manifests + signed record types
  deployments/{devnet,node}/ # reference deployment profiles
  tests/{unit,contract,e2e,chaos}/
  docs/{architecture,protocol,operations,security}/
```

Example lease contract, not a complete wire schema

```
Lease {
  lease_id, content_id, content_version, provider_id,
  manifest_digest, package_bytes, terms_digest,
  start_time, expires_at, state, policy_version
}
AcceptLease {
  signer, offer_id, terms_digest, intent_id
}
```

Specify exact field types, maximum lengths, time units, digest algorithms, signature domains and enum behaviour in the actual Protobuf files. Reuse the SDK transaction-signing envelope rather than inventing signature serialization for chain messages. Publish known-valid and known-invalid fixtures. [S03, S04]

Developer commands to implement

Provide project-owned targets for build, protocol generation, unit tests, devnet start/stop, smoke upload, fault tests and release checks. Names such as make devnet-up are proposed interfaces to create, not commands claimed to exist today. The official example quickstart remains the verified starting reference. [S08]

> **This PDF defines the work and interfaces. It does not contain a complete repository, a runnable genesis file, production keys or a deployable blockchain binary.**

PROJECT CONTROL

26 Risks, decisions and launch checklist

Risk / open decision	Response and launch implication
Blockchain adds little value	Compare with signed records and federated hosting. Retain the chain only for a demonstrated shared-rule requirement.
Too much scope	Finish the media + hosting + custom-chain vertical slice before native apps, tokens or advanced recommendations.
False storage assurance	Label attestations honestly; independent retrieval tests; reviewed proof work is a separate gate.
Concentrated control	Disclose actual operator/council ownership; measure dependency on gateways and account hosts.
Unfunded hosting growth	Hard quotas, accepted lease terms, cost monitoring, renewal and exit procedures.
Privacy or abusive content	Minimize public records, operate moderation, validate removal, and obtain jurisdiction-specific advice.
Licence incompatibility	Pin and review every dependency; exclude Enterprise-only modules unless explicitly licensed.
No creator adoption	Validate one community with permitted content before broad expansion; no adoption forecast is assumed.

Decisions to record before implementation

Confirm product name and target community; custody/recovery model; operator admission; media retention and removal rules; funding and upload allowance; exact dependency versions; software licences; initial council members; and whether the first release permits only adults. None is silently treated as an approved business commitment.

Public-release checklist

The release must have a clean independent install, tested recovery and upgrades, signed builds and source notices, reviewed custom-chain authorization, documented control boundaries, working report/block/removal paths, a privacy and rights policy, measured service targets, a budget owner and named operational coverage.

Final success demonstration

An invited creator publishes an authorized video. Independent hosts accept the package and the chain commits their obligations. The creator and origin disconnect. Another person watches from a surviving host. One provider leaves; replication repairs. A moderator removes a test item from cooperating services. A second developer installs a compatible client. All state and performance claims can be checked.

> **The intended outcome is a usable open-source video network with its own coordination chain. Whether it is cheaper, faster, secure at permissionless scale or commercially viable: I cannot confirm this without implementation and comparative evidence.**

APPENDIX

A Glossary and interpretation

Term	Meaning in this document
Application chain	A blockchain with project-specific execution rules, genesis and validators.
ABCI	The interface through which CometBFT interacts with application logic. [S06]
Consensus	Agreement on ordered transactions and resulting state, under stated fault assumptions. [S05]
Validator	An admitted participant that signs consensus messages; not automatically a storage host.
Provider / host	An operator that accepts media storage or delivery responsibilities.
Attestation	A signed statement by an observer; its truth depends on the observer and verification process.
Manifest / digest	A versioned media inventory and a cryptographic commitment to its exact encoded content.
Lease	A project-defined hosting commitment with content, provider, terms and deadline.
Proof of possession	Evidence under a specified construction that data is held; not automatically separate replication. [S10, S11]
HLS / rendition	HTTP-based adaptive video delivery / one encoded playback variant. [S16]
TURN	A relay mechanism used when a suitable direct peer connection is unavailable. [S15]
Federation	Separate services exchanging permitted records under an agreed protocol. [S14]
Permissioned	Participation in a role is controlled by an admission process.
Permissionless	Admission does not depend on named operator approval; a separate anti-abuse/security design is needed.
RPO / RTO	Recovery point objective / recovery time objective; measure data loss window and restoration time.
Read model	A query-optimized projection that can be reconciled with the authoritative records.

How to read commitments and evidence

Project requirements, architecture decisions, examples and acceptance targets are proposals throughout. Citations support external technical capabilities and policy statements; they do not validate the proposed system. No code was deployed, workload benchmarked or cryptographic scheme audited in preparing this summary.

Research date: October 12, 2026. Software support, service policies and laws can change. Recheck the cited primary sources when pinning dependencies and before public launch. The following source register provides directly inspectable references.

PRIMARY-SOURCE REGISTER

Sources

Sources accessed October 12, 2026. Documentation may change after this date. These references establish external capabilities and constraints, not validation of the proposed implementation.

S01 Cosmos Labs
What is the Cosmos SDK. Application-specific chains, modularity, and Enterprise licence boundary.
https://docs.cosmos.network/sdk/latest/learn/intro/overview

S02 Cosmos Labs
Release Families. Compatibility families and support policy. Versions must be rechecked before a build.
https://docs.cosmos.network/sdk/latest/release-family

S03 Cosmos Labs
Build a Module from Scratch. Protobuf interfaces, keepers, transaction/query handlers, genesis and application wiring.
https://docs.cosmos.network/sdk/latest/tutorials/example/03-build-a-module

S04 Cosmos Labs
Cosmos Architecture. SDK application layers, transaction processing and module composition.
https://docs.cosmos.network/sdk/latest/learn/intro/sdk-app-architecture

S05 CometBFT / Cosmos Labs
What is CometBFT?. Byzantine-fault-tolerant replication and separation of consensus from application logic.
https://docs.cosmos.network/cometbft/latest/docs/introduction/intro

S06 CometBFT / Cosmos Labs
ABCI++ Methods. Application interface, initialization, block execution and validator updates.
https://docs.cosmos.network/cometbft/latest/spec/abci/Methods

S07 CometBFT / Cosmos Labs
Requirements for the Application. Deterministic execution and requirements needed to preserve consensus properties.
https://docs.cosmos.network/cometbft/latest/spec/abci/Requirements-for-the-Application

S08 Cosmos Labs
Chain Quickstart. Reference example chain; tutorial setup is not the proposed product.
https://docs.cosmos.network/sdk/latest/tutorials/example/02-quickstart

PRIMARY-SOURCE REGISTER

Sources / 2

S09 Cosmos Labs
Security and Maintenance Policy. Maintained releases and security reporting.
https://docs.cosmos.network/sdk/latest/security/security-policy

S10 Filecoin
About Proof of Data Possession. Merkle commitments, randomized challenges and Filecoin-specific on-chain verification.
https://docs.filecoin.io/provide-storage/pdp/about

S11 Filecoin
Glossary: proofs and retrieval. Distinction between possession, replication, continued storage and retrieval.
https://docs.filecoin.io/reference/general/glossary

S12 IPFS
Persistence, permanence, and pinning. Retaining copies and paying for persistence; content addressing is not permanent availability.
https://docs.ipfs.tech/concepts/persistence/

S13 IPFS
Privacy and encryption. Public metadata, node identifiability and transport versus content encryption.
https://docs.ipfs.tech/concepts/privacy-and-encryption/

S14 PeerTube / Framasoft
Architecture. Federation, video processing, HTTP/HLS and optional WebRTC delivery.
https://docs.joinpeertube.org/contribute/architecture

S15 WebRTC project
TURN server. Relays for connections that cannot be established directly; self-hosting option.
https://webrtc.org/getting-started/turn-server

S16 Apple Developer
HTTP Live Streaming. Adaptive streaming over ordinary HTTP infrastructure.
https://developer.apple.com/streaming/

PRIMARY-SOURCE REGISTER

Sources / 3

S17 FastAPI
Background Tasks. Heavy computation belongs in separate workers rather than request-process background tasks.
https://fastapi.tiangolo.com/tutorial/background-tasks/

S18 OWASP
File Upload Cheat Sheet. Layered upload validation, quotas, authorization and isolation controls.
https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html

S19 AT Protocol
Protocol Overview. Independent services, feeds and identity/data portability as reference patterns.
https://atproto.com/guides/overview

S20 RFC Editor
RFC 8785: JSON Canonicalization Scheme. Canonical JSON for repeatable hashing/signing. Informational RFC, not Standards Track.
https://www.rfc-editor.org/rfc/rfc8785.html

S21 Google Developers
Recommendation systems overview. Candidate generation, scoring and re-ranking.
https://developers.google.com/machine-learning/recommendation/overview/types

S22 Apple Developer
App Review Guidelines, section 1.2. User-generated-content requirements. Recheck before mobile distribution.
https://developer.apple.com/app-store/review/guidelines/

S23 Google Play
User Generated Content policy. Terms, reporting, blocking and ongoing moderation.
https://support.google.com/googleplay/android-developer/answer/9876937

S24 Office of the Privacy Commissioner of Canada
Summary of privacy laws in Canada. General jurisdictional overview. Page dated January 31, 2018; not a current-law opinion.
https://www.priv.gc.ca/en/privacy-topics/privacy-laws-in-canada/02_05_d_15/

PRIMARY-SOURCE REGISTER

Sources / 4

S25 World Intellectual Property Organization
Copyright. Rights in software, music and audiovisual works; ideas versus expression.
https://www.wipo.int/en/web/copyright

S26 FFmpeg
License and Legal Considerations. Build-dependent LGPL/GPL obligations and codec patent caution.
https://ffmpeg.org/legal.html

S27 Open Source Initiative
GNU Affero General Public License version 3. Licence text, including modified network-interactive versions.
https://opensource.org/license/agpl-3.0

S28 Apache Software Foundation
Apache License, Version 2.0. Permissive licence conditions, notices and patent grant.
https://www.apache.org/licenses/LICENSE-2.0

S29 Loops project
loops-server repository. Existing federated short-video product and AGPL-licensed implementation.
https://github.com/joinloops/loops-server

S30 Livepeer
Run the Livepeer network. Reference for independent operators processing video jobs; not a promised integration.
https://docs.livepeer.org/network

S31 PeerTube / Framasoft
Moderate your PeerTube platform. Operator moderation, reports, account and content controls.
https://docs.joinpeertube.org/admin/moderation